2.i Exceptions

Handling Errors Gracefully in Kotlin

Contents

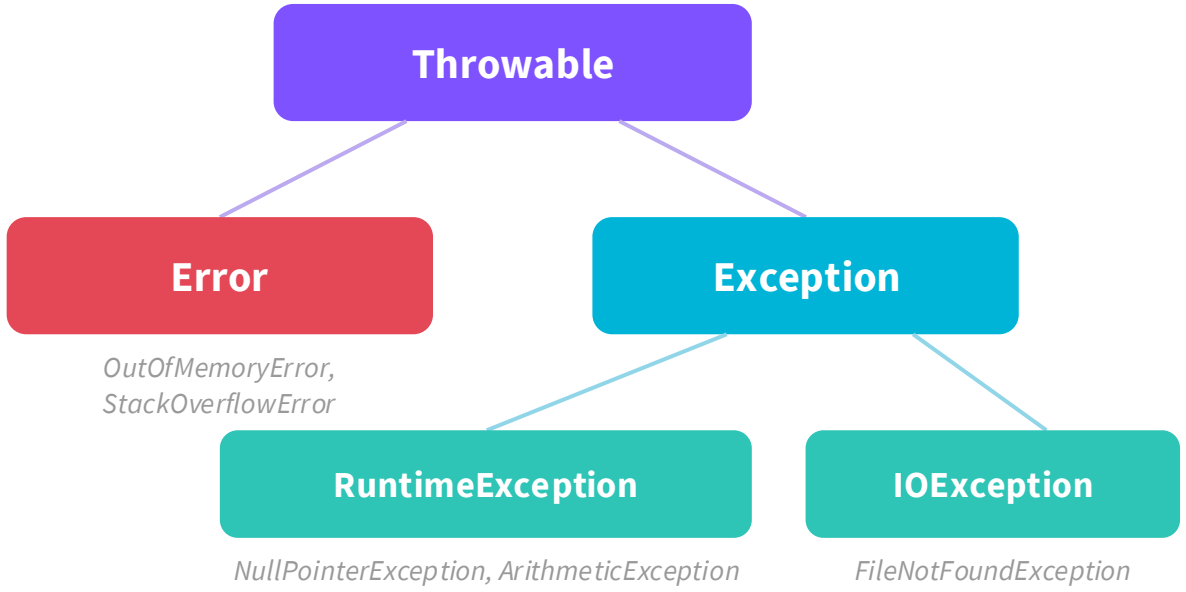
1 **Exception Classes**
Throwable hierarchy, Error vs Exception, all exceptions are unchecked

2 **Throwing Exceptions**
The throw keyword, throw as an expression, and the Nothing type

3 **Catching Exceptions**
try-catch-finally blocks, multiple catch blocks, and try as expression

4 **Custom Exceptions**
Creating domain-specific exception classes and Java interop with @Throws

Exception Classes



Key Concepts

- All exceptions in Kotlin are **unchecked** by default
- Error = unrecoverable system problems; Exception = handleable application conditions
- Exception is an **open** class: create custom exceptions for your needs
- Kotlin prevents common RuntimeExceptions (e.g. NullPointerException) at compile time

Kotlin vs Java: Unlike Java, Kotlin has no checked exceptions. You can catch them but never need to declare them. This simplifies code, especially with lambdas and higher-order functions.

Throwing Exceptions

Basic throw

```
throw IllegalArgumentException(
    "Value must be positive"
)
```

Use **throw** to signal an unexpected runtime error.

Custom exception

```
class CustomException(
    message: String
) : Exception(message)


throw CustomException(
    "Something went wrong"
)
```

throw as Expression

```
val name = person.name
    ?: throw IllegalArgumentException(
        "Name required"
    )
```

throw is an expression with return type **Nothing**.

Precondition functions

```
require(age > 0)
  // throws IllegalArgumentException if arg is true
  // used to validate function arguments

check(isInitialized)
  // throws IllegalStateException if arg is true
  // Used to validate object/program state

error("Fatal error")
  // throws IllegalStateException
  // used to signal an unrecoverable problem
```

Catching Exceptions

try-catch-finally

```
try {
    val result = 10 / 0
} catch (e: ArithmeticException) {
    println(
      "Error: ${e.message}"
    )
} finally {
    println(
      "Cleanup done"
    )
}
```

Multiple catch blocks

```
try {
    val result = 25 / 0
} catch (e: NumberFormatException) {
    // handle format errors
} catch (e: ArithmeticException) {
    // handle math errors
} catch (e: Exception) {
    // catch-all fallback
}
```

Rules

- A try block must have at least one **catch** or **finally** block (or both)
- Order catch blocks from most specific to most general exception type
- The **finally** block always executes, whether an exception was thrown or not

try as an Expression

In Kotlin, **try** is an expression that returns a value. The last expression in the try or catch block becomes the result.

Assign result from try

```
val number: Int = try {
    "123".toInt()
} catch (e: NumberFormatException) {
    0 // default value
}

println(number) // 123
```

→ If parsing succeeds, the Int value is returned; if it throws, the catch block returns 0

With nullable result

```
val input = "abc"

val parsed: Int? = try {
    input.toInt()
} catch (e: NumberFormatException) {
    null
}

println(parsed) // null
```

→ Return null on failure instead of a default value

Note: The finally block does NOT affect the return value of a try expression. Only the try or catch block result is used.

The Nothing Type

Nothing is a special type representing functions or expressions that **never complete successfully**.

Function returning Nothing

```
fun fail(msg: String): Nothing {
    throw IllegalArgumentException(
        msg
    )
}
```

Usage with Elvis operator

```
val name = person.name
    ?: fail(
        "Name required"
    )

// name is smart-cast to non-null
```

How Nothing Works

- Nothing has no instances — a function returning Nothing never returns normally
- The compiler marks code after a Nothing-returning call as unreachable
- Nothing is a subtype of every type, so it can be used where any type is expected
- Use Nothing to mark functions designed to always throw an exception

throw expression return type = **Nothing**

Custom Exceptions & Java Interop

Defining custom exceptions

```
// Simple custom exception
class CustomException(
    message: String = "My message"
) : Exception(message)

// Subclass of existing exception
class NumberTooLargeException
    : ArithmeticException(
        "Number is too large"
    )
```

⚠ Always create a **new instance** each time you throw. Avoid object declarations for exceptions, as the stack trace must reflect the specific context.

@Throws for Java interop

```
// Kotlin function
@Throws(IOException::class)
fun readFile(): String? {
    val path = null
    return path ?: throw
        IOException(
            "Invalid path"
        )
}
```

Since Kotlin has no checked exceptions, Java callers would get a compiler error. The **@Throws** annotation adds the throws clause to the generated JVM bytecode.

Resource management: Kotlin uses **.use { }** instead of Java's try-with-resources. It automatically calls close() after the block.

Practice Exercises

First group

1 Safe Division

Write a function safeDivide(a: Int, b: Int): Int that returns the result of a/b, or 0 if b is zero. Use try-catch.

2 Parse or Default

Write a function parseIntOrDefault(s: String, default: Int): Int using try as an expression.

3 Custom Validation

Create InvalidAgeException. Write validateAge(age: Int) that throws it if age is negative.

Second group

4 File Reader

Write readFileContent(path: String): String that handles FileNotFoundException and returns a meaningful error message.

5 Nothing Helper

Create a function todo(reason: String): Nothing and use it with the Elvis operator on a nullable value.

6 Multi-catch Chain

Write a function that performs string-to-number conversion and division, catching each exception type separately.

Tip: Prefer specific exception types over catching Exception. Use try as an expression for cleaner, more idiomatic Kotlin code.